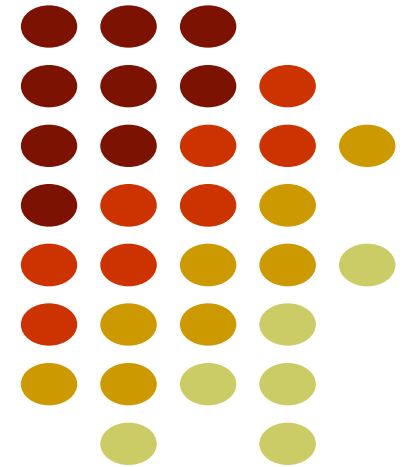


Analyse Syntaxique

Dr. Aida Lahouij
Aida.lahouij@gmail.com



Plan

Introduction

Analyse syntaxique avec BISON

Approche descendante (Top-down)

Approche ascendante (Bottom-up)



Introduction



- Structure générale d'un parser
- Parser = Algorithme qui reconnaît la structure du programme et construit l'arbre syntaxique correspondant
- Comme le langage à reconnaître est context-free, **un parser aura le fonctionnement d'un automate à pile (PDA)**
- Nous verrons 2 grandes classes de parsers :
 - Les analyseurs descendants (top-down)
 - Les analyseurs ascendants (bottom-up)

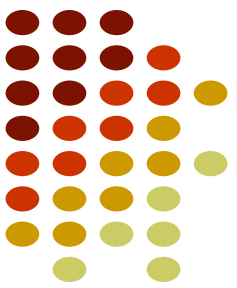
Introduction



Automate à pile:

- Un automate à pile (AP) est une extension d'un automate fini qui dispose d'une pile permettant de mémoriser des informations supplémentaires.
- Il est particulièrement utile pour reconnaître les langages non contextuels, comme les expressions bien parenthésées ou les expressions arithmétiques.

Introduction



Automate à pile:

Un automate à pile est défini par un 7-uplet :

$$AP = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

- Q : Ensemble fini d'états.
- Σ : Alphabet d'entrée.
- Γ : Alphabet de la pile.
- δ : Fonction de transition $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$.
- q_0 : État initial.
- Z_0 : Symbole initial de la pile.
- F : Ensemble d'états finaux (optionnel, dépend de la méthode d'acceptation).

Introduction

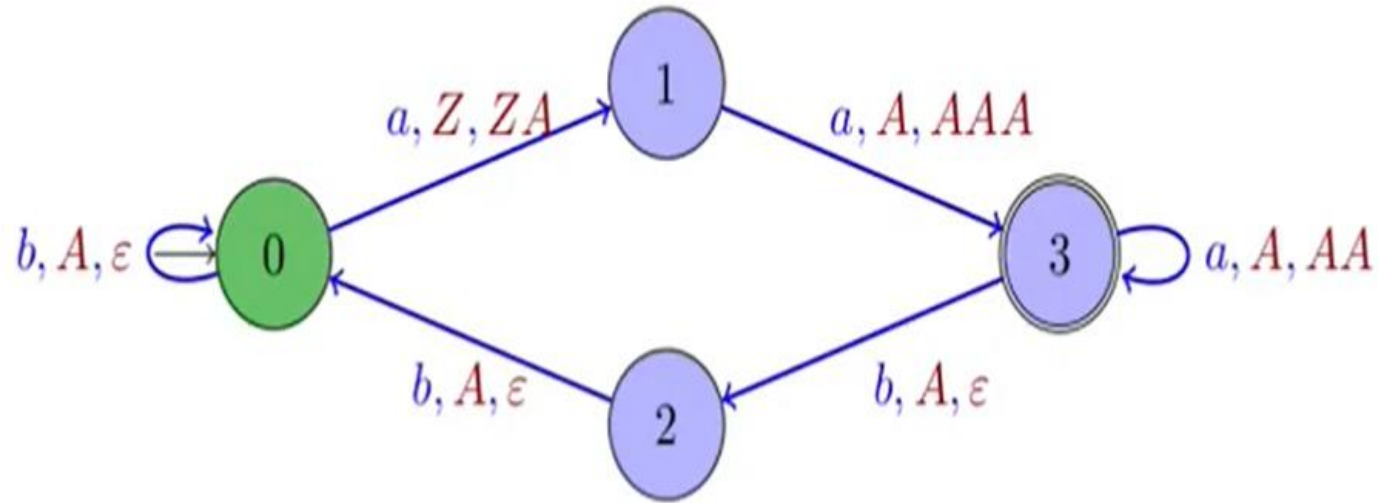


Automate à pile:

- L'automate à pile fonctionne en lisant une entrée symbole par symbole et en effectuant des actions sur la pile selon la transition définie par δ :
 1. Lecture d'un symbole d'entrée (ou ϵ pour une transition sans lecture).
 2. Lecture du sommet de la pile.
 3. Mise à jour de l'état et de la pile :
 - Empiler un symbole.
 - Dépiler un symbole.
 - Remplacer le sommet de la pile.

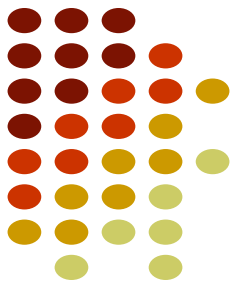
Introduction

Automate à pile:



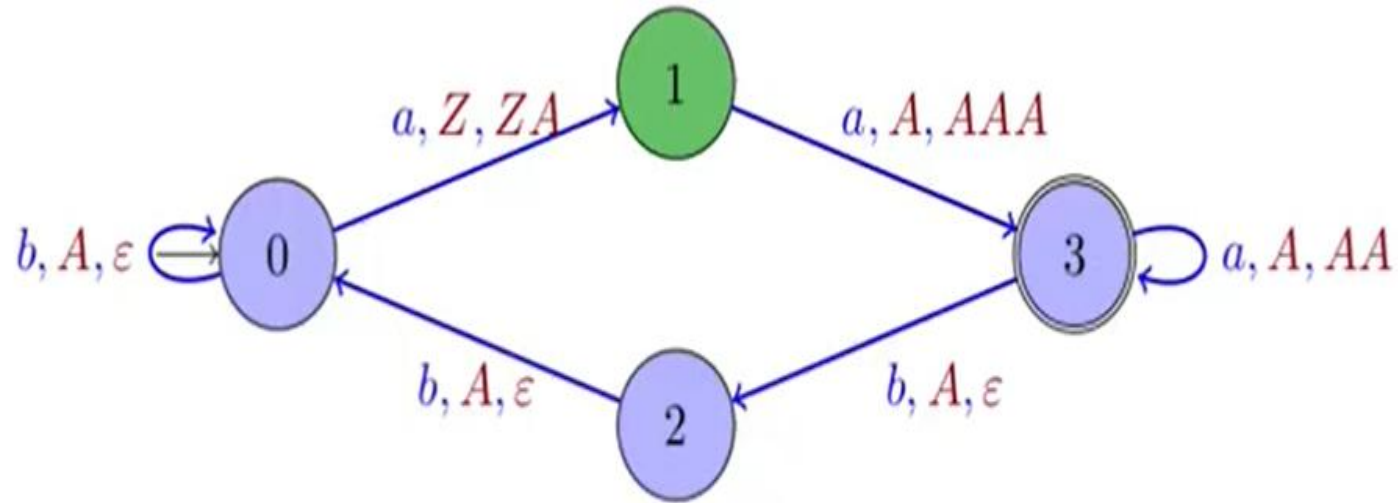
Z

$aaabbbb$



Introduction

Automate à pile:



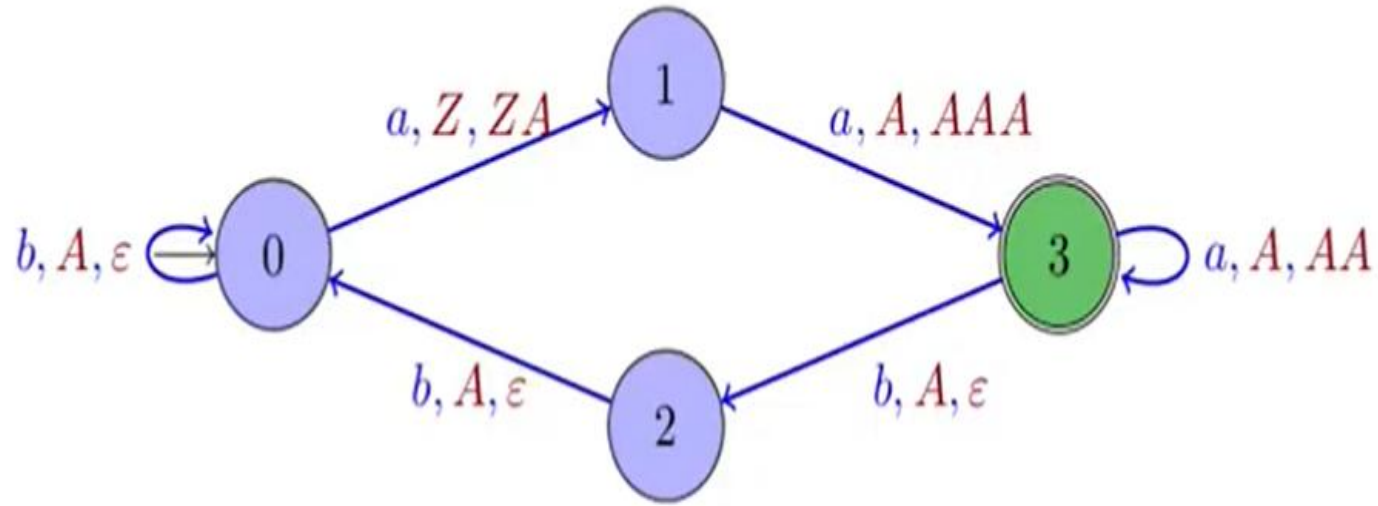
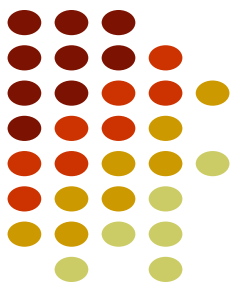
$aabbbb$

A
Z



Introduction

Automate à pile:

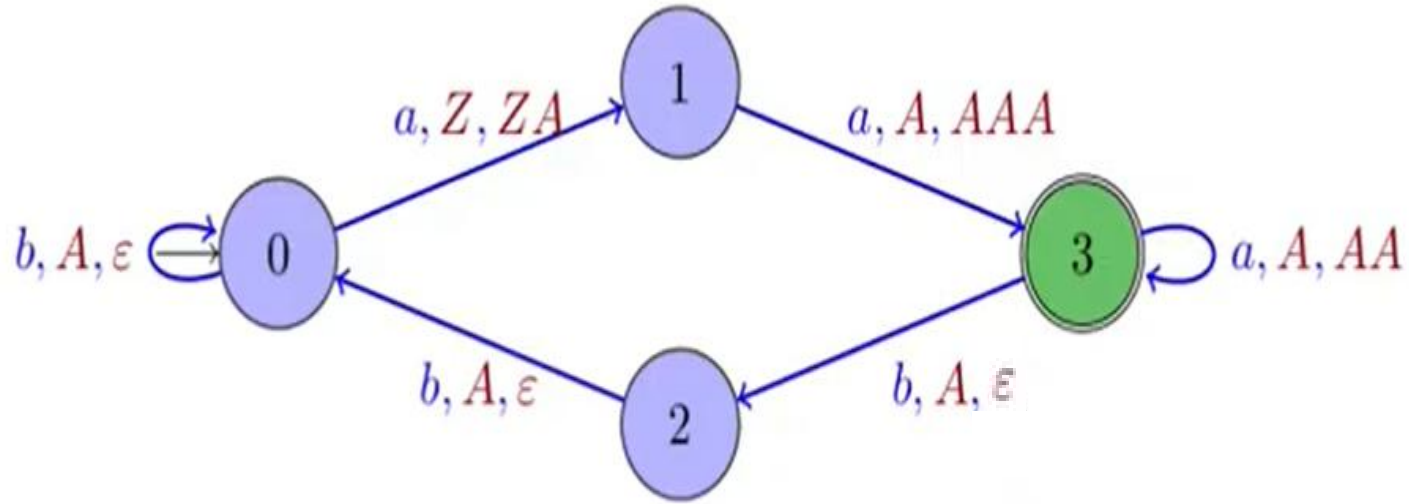


A
A
A
Z

aaabbbb

Introduction

Automate à pile:



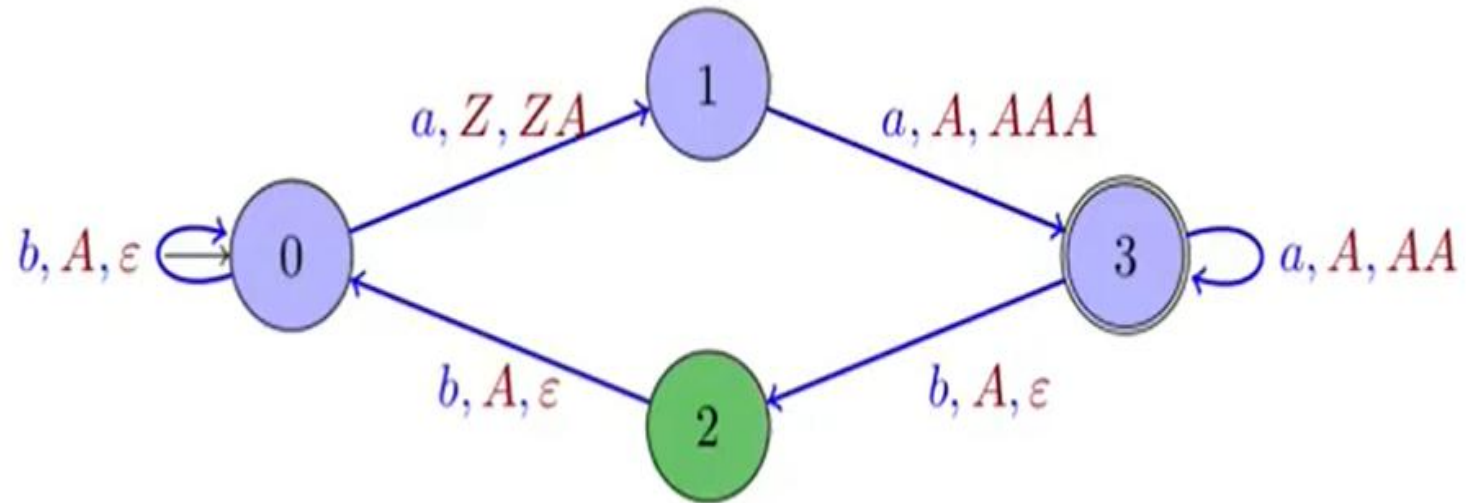
A
A
A
A
Z

aaa**bbb**

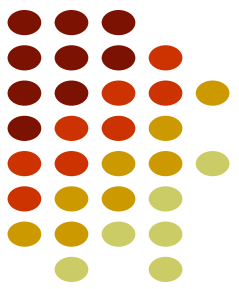


Introduction

Automate à pile:

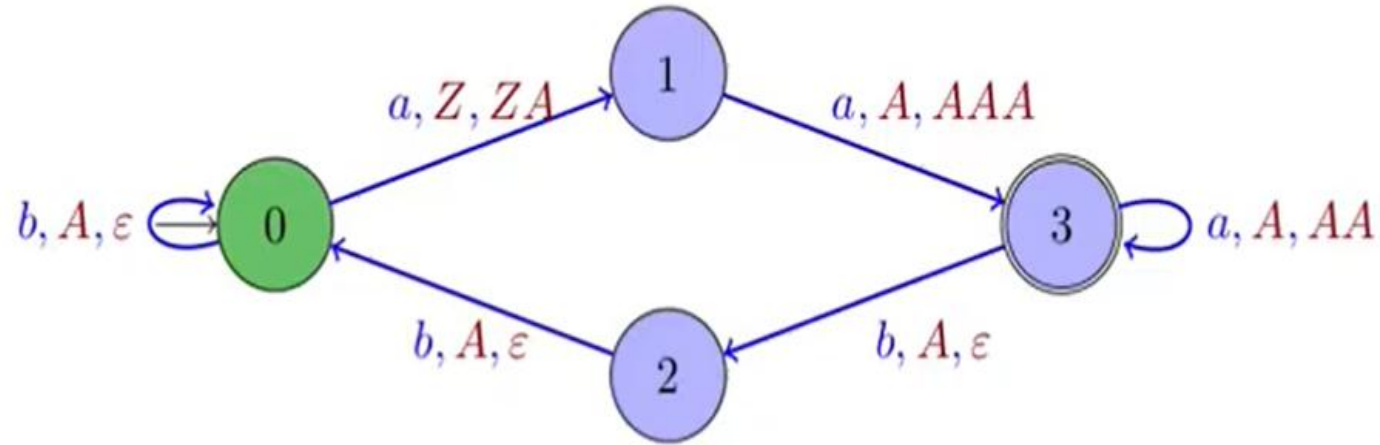
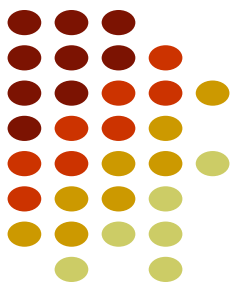


A
A
A
Z



Introduction

Automate à pile:

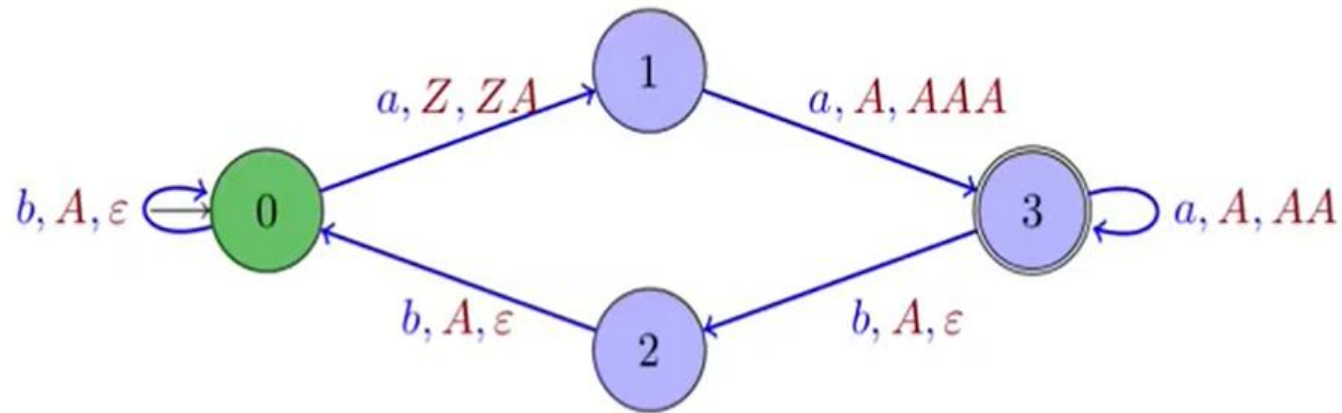
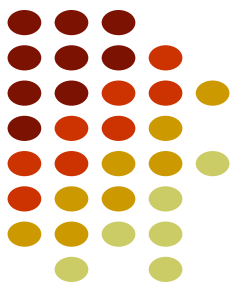


A
A
Z

$aaabbb$

Introduction

Automate à pile:

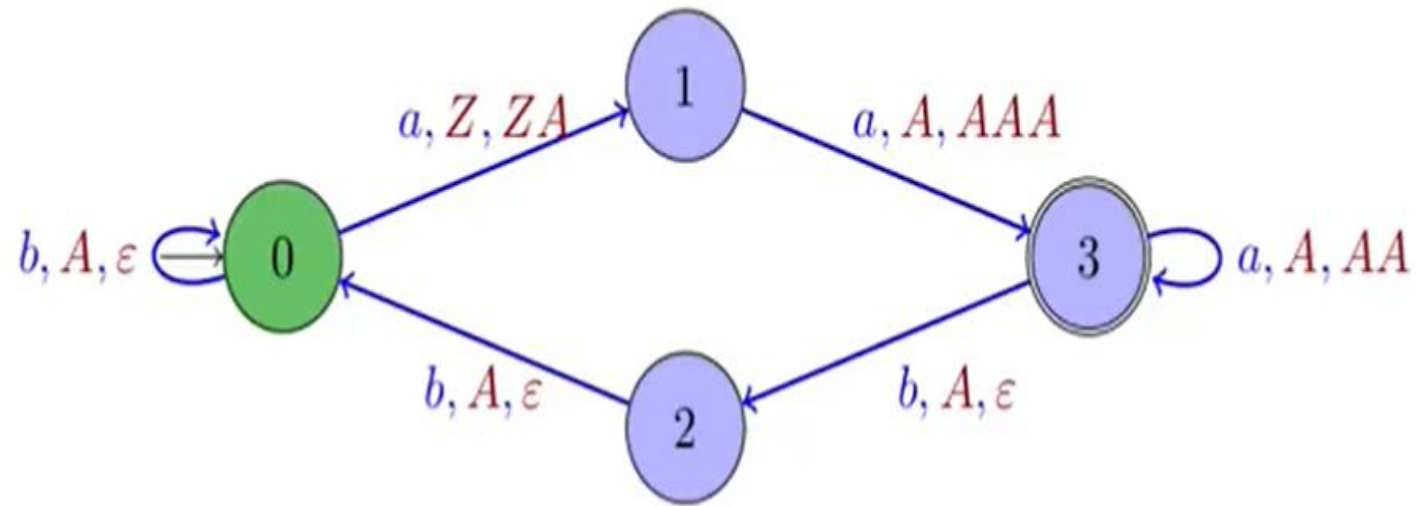


A
Z

$aaabbbb$

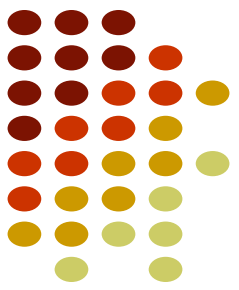
Introduction

Automate à pile:



aaabbbb

Z



Introduction



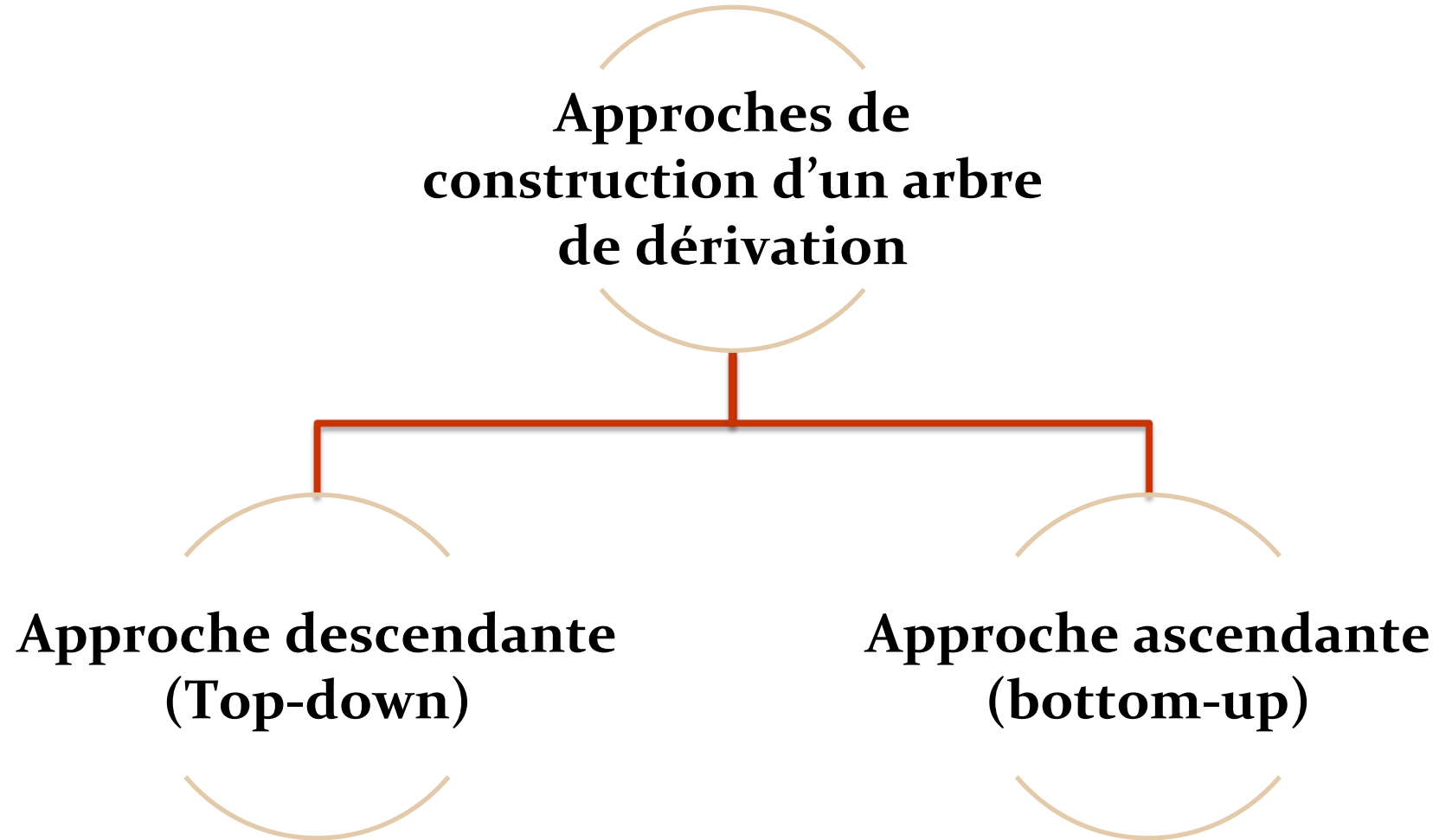
Automate à pile:

Reconnaissance

Il y a trois types de reconnaissance, qui définissent la même classe de langages :

- reconnaissance par pile vide,
- reconnaissance par état final,
- reconnaissance par pile vide et état final.

Introduction



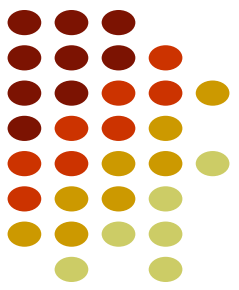


Approche descendante (Top-down)

Approche descendante (Top-down)

Principe

- Construire l'arbre de dérivation du **haut** (la racine, c'est à dire **l'axiome** de départ) vers le **bas** (les feuilles, c'est à dire les **unités lexicales**).
- Le processus d'analyse commence **par la racine de l'arbre** syntaxique et se déplace vers le bas en utilisant les **règles de la grammaire** pour **diviser** la chaîne de caractères en sous-éléments syntaxiques.
- Les règles de la grammaire sont appliquées de manière récursive pour continuer à diviser la chaîne de caractères en sous-éléments jusqu'à ce que les **symboles terminaux** soient atteints.



Approche descendante (Top-down)

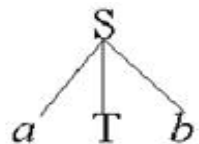
- **Exemple 2**

$S \rightarrow aTb$

$T \rightarrow cd \mid c$

Avec le mot $w = acb$

- Nous partons de S premier non terminal de la grammaire.
- S est la racine de l'arbre. Nous lisons ensuite la première lettre (a) pour arriver à la construction :



$w = \textcolor{brown}{a}cb$



Approche descendante (Top-down)

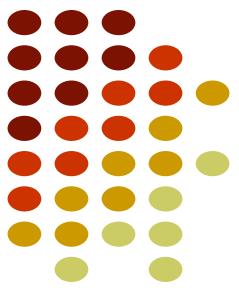
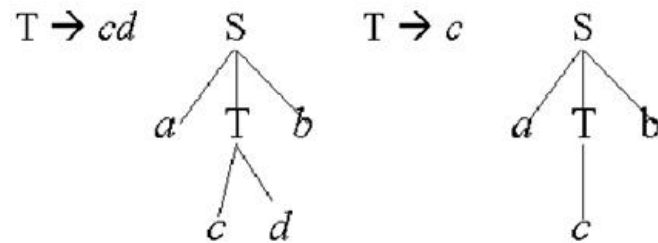
- Exemple

$S \rightarrow aTb$

$T \rightarrow cd \mid c$

Avec le mot $w = acb$

- En lisant le caractère c , nous ne savons pas s'il faut prendre la règle $T \rightarrow cd$ ou la règle $T \rightarrow c$: nous devons essayer une règle et **faire des retours en cas d'échec**.



Approche descendante (Top-down)



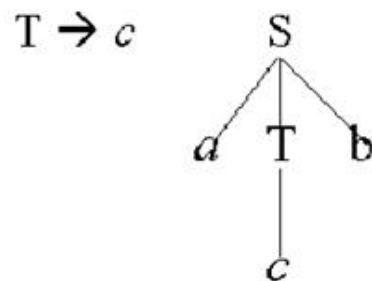
- **Exemple**

$S \rightarrow aTb$

$T \rightarrow cd \mid c$

Avec le mot $w = acb$

- Nous essayons la 1ère règle ($T \rightarrow cd$) qui aboutit à un échec.
- Nous effectuons alors un retour en arrière (backtracking) pour essayer la 2ème règle ($T \rightarrow c$). Dans ce dernier cas le mot $w = acb$ est reconnu.



$w = ac**b**$

Approche descendante (Top-down)



Conclusion

- Ce qui serait pratique, ça serait d'avoir une table qui indique : quand on lit tel caractère et qu'on en est à dériver tel symbole non-terminal, alors on applique telle règle et on ne se pose pas de questions.
- Ça existe, et ça s'appelle **une table d'analyse**.

Approche descendante (Top-down)

Analyseur LL(1)



Définition

- L'analyseur LL(1) est un analyseur syntaxique qui lit l'entrée **de gauche à droite** (Left-to-Right) et construit **une dérivation la plus à gauche** (Leftmost derivation) en utilisant **un seul symbole de lecture anticipée**.

Caractéristiques

- Lecture anticipée : Un seul symbole est examiné à la fois.
- Table LL(1) : Structure permettant de déterminer la production à appliquer selon le symbole en cours.
- Construction basée sur la grammaire : Utilisation des ensembles **First** et **Follow** pour la prédiction.

Approche descendante (Top-down)

Analyseur LL(1)



- Un analyseur **LL(1)** utilise une table d'analyse syntaxique appelée **table d'analyse syntaxique LL(1)**, qui est un tableau de **correspondance entre les symboles de l'entrée et les productions de la grammaire**.
- Cette table est construite à partir de la grammaire du langage que l'on souhaite analyser.
- Elle contient des informations **sur les symboles non terminaux et terminaux, les productions de la grammaire, les premiers et suivants des non terminaux, ainsi que les actions à effectuer lorsque l'analyseur rencontre un symbole dans l'entrée**.



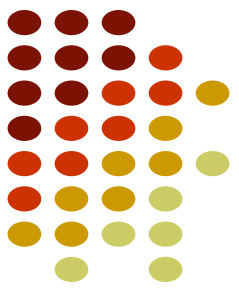
Approche descendante (Top-down)

Analyseur LL(1)

- L'analyseur **LL(1)** effectue une analyse syntaxique en suivant **une stratégie de lecture anticipée de gauche à droite.**
- Il lit un symbole d'entrée et utilise la table d'analyse syntaxique pour déterminer quelle production de la grammaire doit être utilisée pour générer les symboles suivants.
- L'analyseur suit ensuite les symboles de la production et continue de lire l'entrée jusqu'à ce qu'il ait généré une chaîne qui correspond à la phrase donnée en entrée.

Approche descendante (Top-down)

Analyseur LL(1)



- L'analyseur LL(1) a l'**avantage** d'être facile à implémenter et de générer des messages d'erreur clairs lorsqu'il rencontre des erreurs de syntaxe.
- Cependant, il a également des **limitations**, car il ne peut pas analyser toutes les grammaires, en particulier celles qui sont **ambiguës** ou qui nécessitent des décisions de lecture anticipée plus complexes.

Approche descendante (Top-down)

Analyseur LL(1)



- Pour construire une table d'analyse, on a besoin de calculer deux ensembles de terminaux :
 - PREMIER “First()”
 - SUIVANT “Follow()”
- **First** et **Follow** sont des ensembles utilisés dans l'analyse syntaxique pour déterminer quelles productions appliquer lorsqu'on analyse une chaîne de symboles.
- **First(A)** : l'ensemble First(A) contient les terminaux qui peuvent commencer une chaîne de symboles dérivée de A. Par exemple, si A peut produire le symbole B, et si B commence par le terminal a alors First(A) contiendra a.

A → BC
B → a



Approche descendante (Top-down)

Analyseur LL(1)

- **Follow(A)** : l'ensemble Follow(A) contient les terminaux qui peuvent suivre A dans une chaîne de symboles. Par exemple, si A est suivi par B dans une production, et si B peut être suivi par le terminal c, alors Follow(A) contiendra c.
- Ces ensembles sont utilisés pour **construire des tables d'analyse syntaxique LL(1)**, qui permettent de décider quelle production appliquer à chaque étape de l'analyse.

Approche descendante (Top-down)

Analyseur LL(1)

En résumé

L'analyse LL est une analyse syntaxique descendante pour certaines grammaires non contextuelles, dites grammaires LL.

- ▷ **Left to right** : Elle analyse un mot d'entrée de gauche à droite .
- ▷ **Leftmost derivation** : en construit une dérivation à gauche.

L'arbre syntaxique est construit depuis la racine puis en descendant dans l'arbre.

Etapas

- ▷ Calcul des premiers .
- ▷ Calcul des suivants.
- ▷ Construction de la table d'analyse LL(1).
- ▷ Analyse d'un token.





Approche descendante (Top-down)

Calcul des premiers

Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers



Définition ($First^1(\alpha)$ (ou $First(\alpha)$))

$First^1(\alpha)$, dénoté plus simplement $First(\alpha)$, est l'ensemble des symboles terminaux qui peuvent commencer un string généré à partir de α , union ϵ si α peut générer ϵ

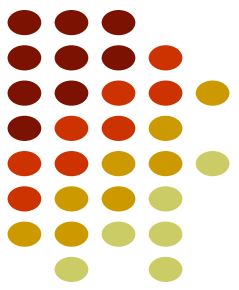
Mathématiquement :

$$First(\alpha) = \begin{aligned} &\{a \in T \mid \exists x \in T^* : \alpha \xrightarrow{*} ax\} \\ &\cup \\ &\{\epsilon \mid \alpha \xrightarrow{*} \epsilon\} \end{aligned}$$

Approche descendante (Top-down)

Analyseur $LL(1)$: Calcul des premiers

Algorithme de calcul des ensembles **PREMIER**(X)



X est composé de terminaux et de non-terminaux :

1. Si X est un non-terminal et $X \rightarrow Y_1 Y_2 \dots Y_n$ est une production de la grammaire (avec Y_i symbole terminal ou non-terminal) alors
 - a. Ajouter les éléments de **PREMIER**(Y_1) sauf ϵ dans **PREMIER**(X)
 - b. Si $\epsilon \in \text{PREMIER}(Y_i)$ pour $i \in [1..j]$ alors **PREMIER**(Y_{i+1}) $\subset$ **PREMIER**(X) sauf ϵ
 - c. Si $\epsilon \in \text{PREMIER}(Y_i)$ pour $i \in [1..n]$ alors $\epsilon \in \text{PREMIER}(X)$
2. Si X est un non-terminal et $X \rightarrow \epsilon$ est une production, alors $\epsilon \in \text{PREMIER}(X)$
3. Si X est un terminal, **PREMIER**(X) = { X } .
4. Recommencer jusqu'à ce qu'on n'ajoute rien de nouveau dans les ensembles **PREMIER**.

Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers



Definition

Pour tout les symboles non terminaux, on définit $\text{Premier}(S)$ comme étant :

L'ensemble des terminaux susceptibles de commencer un mot dérivé de S .

Règles

- ▷ $A \rightarrow \alpha\beta B : \text{Premier}(A) = \{\alpha\}$
- ▷ $A \rightarrow B\alpha, B \rightarrow \beta\sigma C : \text{Premier}(A) = \text{Premier}(B) = \{\beta\}$
- ▷ $A \rightarrow B\alpha, B \rightarrow \beta C \mid \varepsilon :$
 $\text{Premier}(A) = \text{Premier}(B) + \text{Premier}(A \text{ si } B = \varepsilon) = \{\beta, \alpha\}$
 $\text{Premier}(B) = \{\beta, \varepsilon\}$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers



Exemple

Soit la grammaire suivante :

- ▷ $S \rightarrow ABC$
- ▷ $A \rightarrow aA | \varepsilon$
- ▷ $B \rightarrow bB | cB | \varepsilon$
- ▷ $C \rightarrow dC | da | dA$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers

Exemple

Soit la grammaire suivante :

▷ $S \rightarrow ABC$

▷ $A \rightarrow aA | \varepsilon$

▷ $B \rightarrow bB | cB | \varepsilon$

▷ $C \rightarrow dC | da | dA$

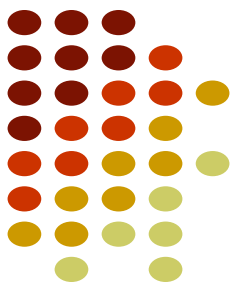
$$\text{premier}(S) = \text{premier}(A) - \varepsilon + \underbrace{\text{premier}(s \text{ si } A=\varepsilon) - \varepsilon}_{\text{premiers (B)} - \varepsilon} + \underbrace{\text{premier}(s \text{ si } A=\varepsilon \text{ et } B=\varepsilon) - \varepsilon}_{\text{premiers (C)} - \varepsilon}$$

$\text{premier}(A) = \{a, \varepsilon\}$

$\text{premiers (B)} = \{b, c, \varepsilon\}$

$\text{premiers (C)} = \{d\}$

⇒ $\text{premier}(S) = \{a, b, c, d\}$



Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers



Exemple

Soit la grammaire suivante :

- ▷ $S \rightarrow ABC$
- ▷ $A \rightarrow aA | \varepsilon$
- ▷ $B \rightarrow bB | cB | \varepsilon$
- ▷ $C \rightarrow dC | da | dA$

Les premiers

- ▷ $\text{Premier}(S) = \{a, b, c, d\}$
- ▷ $\text{Premier}(A) = \{a, \varepsilon\}$
- ▷ $\text{Premier}(B) = \{b, c, \varepsilon\}$
- ▷ $\text{Premier}(C) = \{d\}$

Approche descendante (Top-down)

Analyseur $LL(1)$: Calcul des premiers

Si on rajoute ϵ à la règle de production de C

Exemple

Soit la grammaire suivante :

▷ $S \rightarrow ABC$

▷ $A \rightarrow aA | \epsilon$

▷ $B \rightarrow bB | cB | \epsilon$

▷ $C \rightarrow dC | da | dA | \epsilon$

$\text{premier}(A) = \{a, \epsilon\}$

$\text{premiers}(B) = \{b, c, \epsilon\}$

$\text{premiers}(C) = \{d, \epsilon\}$

$\text{premier}(S) = \text{premier}(A) - \epsilon + \text{premier}(S \text{ si } A=\epsilon) - \epsilon + \text{premier}(S \text{ si } A=\epsilon \text{ et } B=\epsilon) - \epsilon$
 $+ \text{premier}(S \text{ si } A=\epsilon \text{ et } B=\epsilon \text{ et } C=\epsilon)$

$= \text{premier}(A) - \epsilon + \text{premiers}(B) - \epsilon + \text{premier}(C) - \epsilon + \epsilon$

➡ $\text{premier}(S) = \{a, b, c, d, \epsilon\}$



Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers



• Calcul des ensembles PREMIER(X) : Exemple

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid -TE' \mid \varepsilon$

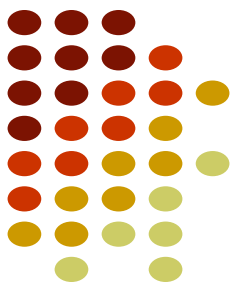
$T \rightarrow FT'$

$T' \rightarrow *FT' \mid /FT' \mid \varepsilon$

$F \rightarrow (E) \mid nb$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers



• Calcul des ensembles PREMIER(X) : Exemple

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid -TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid /FT' \mid \varepsilon$

$F \rightarrow (E) \mid nb$

$\underline{a} \in \text{PREMIER}(\alpha)$ si et seulement si : $\alpha \rightarrow \underline{a}\beta$

- $\text{PREMIER}(E)$
- $\text{PREMIER}(T)$
- $\text{PREMIER}(E') = \{+, -\}$
- $\text{PREMIER}(F) = \{ (, nb \}$
- $\text{PREMIER}(T') = \{ *, / \}$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers



• Calcul des ensembles PREMIER(X) : Exemple

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid -TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid /FT' \mid \epsilon$

$F \rightarrow (E) \mid nb$

2. Si X est un non-terminal et $X \rightarrow \epsilon$ une production, alors $\epsilon \in \text{PREMIER}(X)$

- $\text{PREMIER}(E)$
- $\text{PREMIER}(T)$
- $\text{PREMIER}(E') = \{+, -, \epsilon\}$
- $\text{PREMIER}(F) = \{ (, nb \}$
- $\text{PREMIER}(T') = \{ *, /, \epsilon \}$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers



• Calcul des ensembles PREMIER(X) : Exemple

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid -TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid /FT' \mid \varepsilon$

$F \rightarrow (E) \mid nb$

1. Si X est un non-terminal et $X \rightarrow Y_1Y_2...Y_n$ avec Y_i symbole terminal ou non-terminal alors :

1. Ajouter à l'ensemble PREMIER(X) tous les

- $\text{PREMIER}(E) = \text{PREMIER}(T) - \{\varepsilon\} = \{(, nb\}$ éléments de $\text{PREMIER}(Y_1)$ sauf ε
- $\text{PREMIER}(T) = \text{PREMIER}(F) - \{\varepsilon\} = \{(, nb\}$
- $\text{PREMIER}(E') = \{+, -, \varepsilon\}$
- $\text{PREMIER}(F) = \{(, nb\}$
- $\text{PREMIER}(T') = \{*, /, \varepsilon\}$

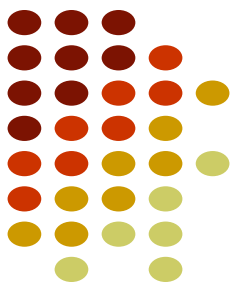
Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers

Exemple (de calcul de $First(A)$ ($\forall A \in V$) avec $G = \langle V, T, P, S' \rangle$)

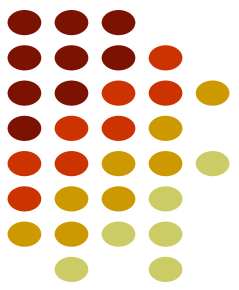
où P contient :

- $S' \rightarrow ES$
- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid id$



Approche descendante (Top-down)

Analyseur LL(1): Calcul des premiers



Exemple (de calcul de $First(A)$ ($\forall A \in V$) avec $G = \langle V, T, P, S' \rangle$)

où P contient :

- $S' \rightarrow E\$$
- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid id$
- $First(S') = \{id, (\}$
- $First(E) = \{id, (\}$
- $First(E') = \{+, \epsilon\}$
- $First(T) = \{id, (\}$
- $First(T') = \{*, \epsilon\}$
- $First(F) = \{id, (\}$



Approche descendante (Top-down)

Calcul des suivants

Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants

- Pour tout non-terminal A, on cherche SUIVANT(A) :

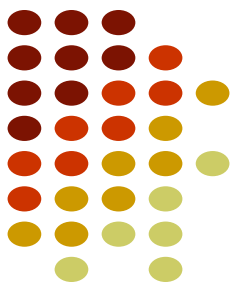
SUIVANT(A) = l'ensemble de tous les symboles terminaux **a** qui peuvent apparaître immédiatement à droite de A dans une dérivation $S \rightarrow \alpha A \mathbf{a} \beta$

Définition ($Follow^k(\beta)$) Pour la CFG G, un entier positif k et $\beta \in (T \cup V)^*$

$Follow^k(\beta)$ est l'ensemble des strings de terminaux de longueur maximale mais limitée à k caractères qui peuvent **suivre** un string généré à partir de β

Mathématiquement :

$$Follow^k(\beta) = \{w \in T^{\leq k} \mid \exists \alpha, \gamma \in (T \cup V)^* : S' \xRightarrow{*} \alpha \beta \gamma \wedge w \in First^k(\gamma)\}$$

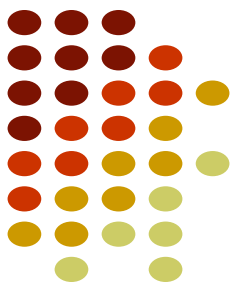


Approche descendante (Top-down)

Analyseur $LL(1)$: Calcul des suivants

Algorithme de calcul des ensembles SUIVANT(X)

1. Ajouter un marqueur de fin de chaîne (symbole \$ par exemple) à **SUIVANT(S)** (où S est l'axiome de départ de la grammaire)
2. S'il y a une production $A \rightarrow \alpha BC$ où B est un non-terminal, alors ajouter le contenu de **PREMIER(C)** à **SUIVANT(B)**, sauf ϵ
3. S'il y a une production $A \rightarrow \alpha B$, alors ajouter **SUIVANT(A)** à **SUIVANT(B)**
4. S'il y a une production $A \rightarrow \alpha BC$ avec $\epsilon \in \text{PREMIER}(C)$, alors ajouter **SUIVANT(A)** à **SUIVANT(B)**
5. Recommencer à partir de l'étape 3 jusqu'à ce qu'on n'ajoute rien de nouveau dans les ensembles **SUIVANT**.



Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants



Definition

Pour tout les symboles non terminaux, on définit $Suivant(S)$ comme étant :

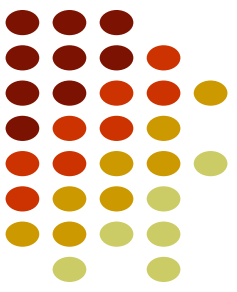
L'ensemble des terminaux susceptibles de suivre un mot dérivé de S

Règles

Soit la grammaire suivante :

$$\begin{aligned} \triangleright A &\rightarrow BC & C &\rightarrow \alpha BC | \varepsilon \\ \triangleright B &\rightarrow DE & E &\rightarrow \beta DE | \varepsilon & D &\rightarrow \gamma A \sigma | \theta \end{aligned}$$

- $Suivant(A) = \{\sigma, \$\}$
- $Suivant(E) = Suivant(B) = \{\alpha, \sigma, \$\}$
- $Suivant(D) = Premier(E) = \{\beta\} \cup Suivant(B) = \{\beta, \alpha, \sigma, \$\}$
 - $Suivant(B) = premier(C) - \varepsilon + Suivant(A \text{ si } C = \varepsilon) + Suivant(C \text{ si } C \neq \varepsilon) = \{\alpha, \sigma, \$\}$
 - $Suivant(C) = Suivant(A) + Suivant(C) = Suivant(A) = \{\sigma, \$\}$



Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants

Definition

Pour tout les symboles non terminaux, on définit $Suivant(S)$ comme étant :

L'ensemble des terminaux susceptibles de suivre un mot dérivé de S

Règles

Soit la grammaire suivante :

$$\begin{array}{lll} \triangleright A \rightarrow BC & C \rightarrow \alpha BC | \varepsilon & \\ \triangleright B \rightarrow DE & E \rightarrow \beta DE | \varepsilon & D \rightarrow \gamma \boxed{A} \sigma | \theta \end{array}$$

- $Suivant(A) = \{\sigma, \$\}$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants

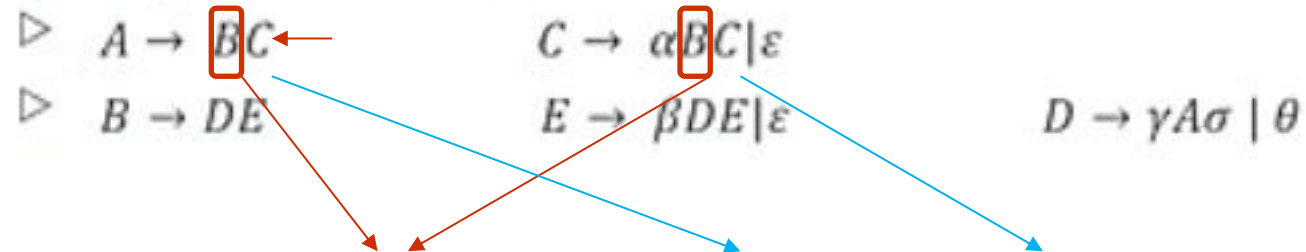
Definition

Pour tout les symboles non terminaux, on définit $Suivant(S)$ comme étant :

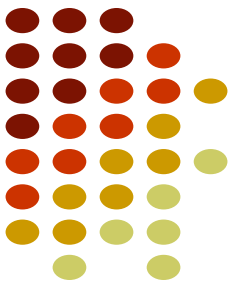
L'ensemble des terminaux susceptibles de suivre un mot dérivé de S

Règles

Soit la grammaire suivante :



- $Suivant(B) = premier(C) - \epsilon + Suivant(A \text{ si } C = \epsilon) + Suivant(C \text{ si } C = \epsilon)$



Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants

Definition

Pour tout les symboles non terminaux, on définit $Suivant(S)$ comme étant :

L'ensemble des terminaux susceptibles de suivre un mot dérivé de S

Règles

Soit la grammaire suivante :

$$\begin{array}{ll} \triangleright A \rightarrow BC & C \rightarrow \alpha BC | \epsilon \\ \triangleright B \rightarrow DE & E \rightarrow \beta DE | \epsilon \end{array} \quad D \rightarrow \gamma A \sigma | \theta$$

- $Suivant(C) = Suivant(A) + Suivant(C) = \{\sigma, \$\}$



Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants

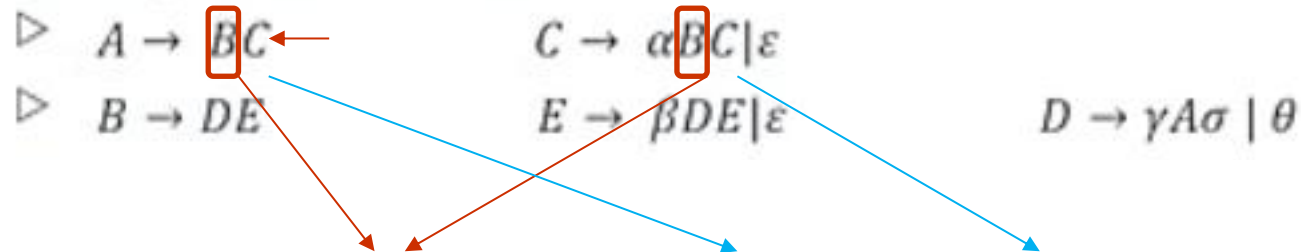
Definition

Pour tout les symboles non terminaux, on définit $Suivant(S)$ comme étant :

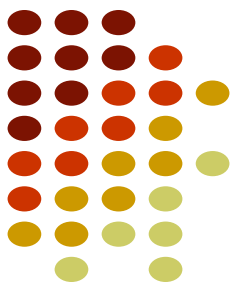
L'ensemble des terminaux susceptibles de suivre un mot dérivé de S

Règles

Soit la grammaire suivante :

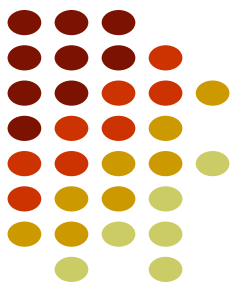


- $Suivant(B) = premier(C) - \epsilon + Suivant(A \text{ si } C = \epsilon) + Suivant(C \text{ si } C = \epsilon)$
 $= \{\alpha, \sigma, \$\}$



Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants



Definition

Pour tout les symboles non terminaux, on définit $Suivant(S)$ comme étant :

L'ensemble des terminaux susceptibles de suivre un mot dérivé de S

Règles

Soit la grammaire suivante :

▷ $A \rightarrow BC$ $C \rightarrow \alpha BC | \epsilon$

▷ $B \rightarrow DE$ $E \rightarrow \beta DE | \epsilon$

$D \rightarrow \gamma A \sigma | \theta$

- $Suivant(E) = Suivant(B) + Suivant(E)$
 $= Suivant(B)$
 $= \{\alpha, \sigma, \$\}$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants



Definition

Pour tout les symboles non terminaux, on définit $Suivant(S)$ comme étant :

L'ensemble des terminaux susceptibles de suivre un mot dérivé de S

Règles

Soit la grammaire suivante :

$$\begin{array}{lll} \triangleright A \rightarrow BC & C \rightarrow \alpha BC | \varepsilon & \\ \triangleright B \rightarrow DE & E \rightarrow \beta DE | \varepsilon & D \rightarrow \gamma A \sigma | \theta \end{array}$$

- $Suivant(D) = premier(E) - \varepsilon + Suivant(B \text{ si } E = \varepsilon) + Suivant(E \text{ si } E = \varepsilon)$
 $= \{\beta, \alpha, \sigma, \$\}$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants

Definition

Pour tout les symboles non terminaux, on définit *Suivant*(*S*) comme étant :

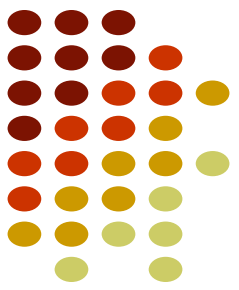
L'ensemble des terminaux susceptibles de suivre un mot dérivé de *S*

Règles

Soit la grammaire suivante :

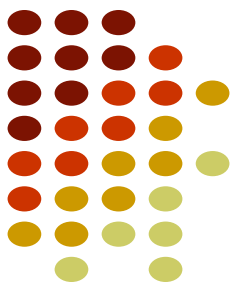
$$\begin{array}{lll} \triangleright A \rightarrow BC & C \rightarrow \alpha BC | \varepsilon & \\ \triangleright B \rightarrow DE & E \rightarrow \beta DE | \varepsilon & D \rightarrow \gamma A \sigma | \theta \end{array}$$

- $\text{Suivant}(A) = \{\sigma, \$\}$
- $\text{Suivant}(B) = \{\alpha, \sigma, \$\}$
- $\text{Suivant}(C) = \{\sigma, \$\}$
- $\text{Suivant}(D) = \{\beta, \alpha, \sigma, \$\}$
- $\text{Suivant}(E) = \{\alpha, \sigma, \$\}$



Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants



Exemple

Soit la grammaire suivante :

- ▷ $S \rightarrow ABC$
- ▷ $A \rightarrow aA | \varepsilon$
- ▷ $B \rightarrow bB | cB | \varepsilon$
- ▷ $C \rightarrow dC | da | dA$

Les premiers

- ▷ $Premier(S) = \{a, b, c, d\}$
- ▷ $Premier(A) = \{a, \varepsilon\}$
- ▷ $Premier(B) = \{b, c, \varepsilon\}$
- ▷ $Premier(C) = \{d\}$

Suivant(S)={ $\$$ }

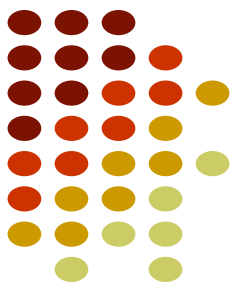
Suivant(A)=Premier(B)- ε + Premier(C si B= ε)- ε +Suivant(A) +Suivant(C)={b,c,d, $\$$ }

Suivant(C)=Suivant(S)+Suivant(C)={ $\$$ }

Suivant(B)= Premier(C)- ε +Suivant(B)={d}

Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants



Exemple

Soit la grammaire suivante :

- ▷ $S \rightarrow ABC$
- ▷ $A \rightarrow aA | \varepsilon$
- ▷ $B \rightarrow bB | cB | \varepsilon$
- ▷ $C \rightarrow dC | da | dA$

Les premiers

- ▷ $Premier(S) = \{a, b, c, d\}$
- ▷ $Premier(A) = \{a, \varepsilon\}$
- ▷ $Premier(B) = \{b, c, \varepsilon\}$
- ▷ $Premier(C) = \{d\}$

Les suivants

- ▷ $Suivant(S) = \{\$ \}$
- ▷ $Suivant(A) = \{b, c, d, \$ \}$
- ▷ $Suivant(B) = \{d\}$
- ▷ $Suivant(C) = \{\$ \}$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants

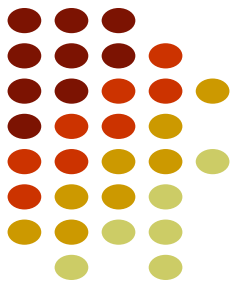
Exemple

$S \rightarrow iEtSS' | a$

$S' \rightarrow eS | \varepsilon$

$E \rightarrow bb$

	PREMIER	SUIVANT
S		
S'		
E		



Approche descendante (Top-down)

Analyseur $LL(1)$: Calcul des suivants

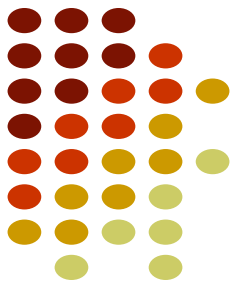
Exemple

$S \rightarrow iEtSS' | a$

$S' \rightarrow eS | \epsilon$

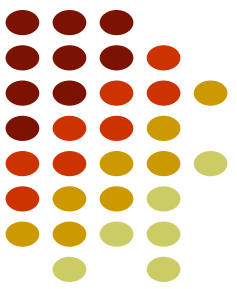
$E \rightarrow bb$

	PREMIER	SUIVANT
S	i a	\$ e
S'	e ϵ	\$ e
E	b	t



Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants



Exemple (de calcul de $Follow(A)$ ($\forall A \in V$) avec $G = \langle V, T, P, S' \rangle$)

dont les règles sont :

- $S' \rightarrow E\$$
- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid id$

On obtient en appliquant l'algorithme (voir figure) :

- $First(S') = \{id, (\}$
- $First(E) = \{id, (\}$
- $First(E') = \{+, \epsilon\}$
- $First(T) = \{id, (\}$
- $First(T') = \{*, \epsilon\}$
- $First(F) = \{id, (\}$

Approche descendante (Top-down)

Analyseur LL(1): Calcul des suivants



Exemple (de calcul de $Follow(A)$ ($\forall A \in V$) avec $G = \langle V, T, P, S' \rangle$)

dont les règles sont :

- $S' \rightarrow E\$$
- $E \rightarrow TE'$
- $E' \rightarrow +TE' \mid \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow *FT' \mid \epsilon$
- $F \rightarrow (E) \mid id$

On obtient en appliquant l'algorithme (voir figure) :

- $Follow(S') = \{\$ \}$
- $Follow(E) = \{), \$ \}$
- $Follow(E') = \{), \$ \}$
- $Follow(T) = \{), \$, + \}$
- $Follow(T') = \{), \$, + \}$
- $Follow(F) = \{), \$, +, * \}$
- $First(S') = \{id, (\}$
- $First(E) = \{id, (\}$
- $First(E') = \{+, \epsilon \}$
- $First(T) = \{id, (\}$
- $First(T') = \{*, \epsilon \}$
- $First(F) = \{id, (\}$



Approche descendante (Top-down)

Construction de la table d'analyse

Approche descendante (Top-down)

Analyseur $LL(1)$: Construction de la table d'analyse



- Une table d'analyse est **une matrice M** où :
 - Dans la **première colonne**, on représente **les non-terminaux**.
 - Dans la **première ligne** on place **les terminaux** et le symbole $\$$ (sauf le mot vide).
 - Chaque case $M[X,x]$ comporte la règle de production à appliquer.

Approche descendante (Top-down)

Analyseur $LL(1)$: Construction de la table d'analyse

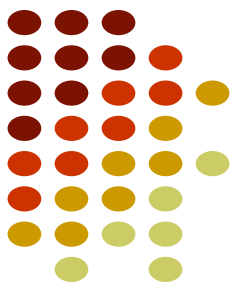


- **Algorithme de construction de la table d'analyse**

1. Pour chaque production $A \rightarrow \alpha$ faire
 - a. pour tout $a \in \text{PREMIER}(\alpha)$ (et $a \neq \epsilon$), rajouter la production $A \rightarrow \alpha$ dans la case $M[A, a]$.
 - b. si $\epsilon \in \text{PREMIER}(\alpha)$, alors pour chaque $b \in \text{SUIVANT}(A)$ ajouter $A \rightarrow \alpha$ dans $M[A, b]$.
2. Chaque case $M[A, a]$ vide est une erreur de syntaxe.

Approche descendante (Top-down)

Analyseur LL(1): Construction de la table d'analyse



Soit la grammaire suivante :

- ▷ $S \rightarrow ABC$
- ▷ $A \rightarrow aA | \varepsilon$
- ▷ $B \rightarrow bB | cB | \varepsilon$
- ▷ $C \rightarrow dC | da | dA$

	Premier	Suivant
S	$\{a, b, c, d\}$	$\{\$ \}$
A	$\{a, \varepsilon\}$	$\{b, c, d, \$ \}$
B	$\{b, c, \varepsilon\}$	$\{d\}$
C	$\{d\}$	$\{\$ \}$

Structure

	Symboles terminaux				
Symboles Non terminaux	a	b	c	d	\$
S					
A					
B					
C					

Approche descendante (Top-down)

Analyseur LL(1): Construction de la table d'analyse



Application de l'algorithme :

► Pour $S \rightarrow ABC$:

$PREMIER(ABC) = PREMIER(A) = \{a, \epsilon\}$

→ on regarde aussi $PREMIER(B) = \{b, c, \epsilon\}$

→ puis $PREMIER(C) = \{d\}$

Donc : $PREMIER(ABC) = \{a, b, c, d\}$

✓ Ajout dans les cases :

| $M[S, a]$ | = $S \rightarrow ABC$
| $M[S, b]$ | = $S \rightarrow ABC$
| $M[S, c]$ | = $S \rightarrow ABC$
| $M[S, d]$ | = $S \rightarrow ABC$

► Pour $A \rightarrow aA$:

$PREMIER(aA) = \{a\}$

✓ $M[A, a] = A \rightarrow aA$

► Pour $A \rightarrow \epsilon$:

$\epsilon \in PREMIER(\epsilon)$, donc on regarde $SUIVANT(A) = \{b, c, d, \$\}$

✓ Ajouts :

$M[A, b] = A \rightarrow \epsilon$

$M[A, c] = A \rightarrow \epsilon$

$M[A, d] = A \rightarrow \epsilon$

$M[A, \$] = A \rightarrow \epsilon$

► Pour $B \rightarrow bB$:

$PREMIER(bB) = \{b\}$

✓ $M[B, b] = B \rightarrow bB$

► Pour $B \rightarrow cB$:

$PREMIER(cB) = \{c\}$

✓ $M[B, c] = B \rightarrow cB$

► Pour $B \rightarrow \epsilon$:

$\epsilon \in PREMIER(\epsilon) \rightarrow$ on regarde $SUIVANT(B) = \{d\}$

✓ $M[B, d] = B \rightarrow \epsilon$

► Pour $C \rightarrow dC$:

$PREMIER(dC) = \{d\}$

✓ $M[C, d] = C \rightarrow dC$

(Note: il y aura conflit ici avec les deux autres règles car elles ont toutes $PREMIER = \{d\}$)

► Pour $C \rightarrow da$:

$PREMIER(da) = \{d\}$

✓ $M[C, d] = C \rightarrow da \leftarrow$ conflit LL(1)

► Pour $C \rightarrow dA$:

$PREMIER(dA) = \{d\}$

✓ $M[C, d] = C \rightarrow dA \leftarrow$ conflit LL(1)

Approche descendante (Top-down)

Analyseur LL(1): Construction de la table d'analyse



Soit la grammaire suivante :

- ▷ $S \rightarrow ABC$
- ▷ $A \rightarrow aA | \varepsilon$
- ▷ $B \rightarrow bB | cB | \varepsilon$
- ▷ $C \rightarrow dC | da | dA$

	Premier	Suivant
S	$\{a, b, c, d\}$	$\{\$ \}$
A	$\{a, \varepsilon\}$	$\{b, c, d, \$ \}$
B	$\{b, c, \varepsilon\}$	$\{d\}$
C	$\{d\}$	$\{\$ \}$

Structure

	Symboles terminaux				
Symboles Non terminaux	a	b	c	d	\$
S					
A					
B					
C					

Approche descendante (Top-down)

Analyseur LL(1)



Soit la grammaire suivante :

- ▷ $S \rightarrow ABC$
- ▷ $A \rightarrow aA | \epsilon$
- ▷ $B \rightarrow bB | cB | \epsilon$
- ▷ $C \rightarrow dC | da | dA$

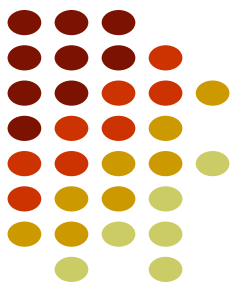
	Premier	Suivant
S	$\{a, b, c, d\}$	$\{\$ \}$
A	$\{a, \epsilon\}$	$\{b, c, d, \$ \}$
B	$\{b, c, \epsilon\}$	$\{d\}$
C	$\{d\}$	$\{\$ \}$

Structure

	Symboles terminaux				
Symboles Non terminaux	a	b	c	d	\$
S	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	
A					
B					
C					

Approche descendante (Top-down)

Analyseur LL(1)



Soit la grammaire suivante :

▷ $S \rightarrow ABC$

▷ $A \rightarrow aA | \epsilon$

▷ $B \rightarrow bB | cB | \epsilon$

▷ $C \rightarrow dC | da | dA$

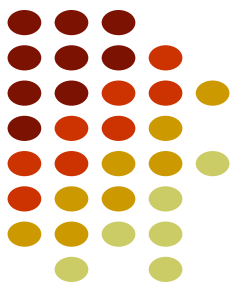
	Premier	Suivant
S	$\{a, b, c, d\}$	$\{\$ \}$
A	$\{a, \epsilon\}$	$\{b, c, d, \$ \}$
B	$\{b, c, \epsilon\}$	$\{d\}$
C	$\{d\}$	$\{\$ \}$

Structure

	Symboles terminaux				
Symboles Non terminaux	a	b	c	d	\$
S	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	
A	$A \rightarrow aA$				
B					
C					

Approche descendante (Top-down)

Analyseur LL(1)



Soit la grammaire suivante :

- ▷ $S \rightarrow ABC$
- ▷ $A \rightarrow aA | \epsilon$
- ▷ $B \rightarrow bB | cB | \epsilon$
- ▷ $C \rightarrow dC | da | dA$

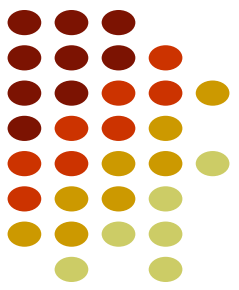
	Premier	Suivant
S	$\{a, b, c, d\}$	$\{\$ \}$
A	$\{a, \epsilon\}$	$\{b, c, d, \$ \}$
B	$\{b, c, \epsilon\}$	$\{d\}$
C	$\{d\}$	$\{\$ \}$

Structure

	Symboles terminaux				
Symboles Non terminaux	a	b	c	d	\$
S	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	
A	$A \rightarrow aA$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
B					
C					

Approche descendante (Top-down)

Analyseur LL(1)



Soit la grammaire suivante :

▷ $S \rightarrow ABC$

▷ $A \rightarrow aA | \epsilon$

▷ $B \rightarrow bB | cB | \epsilon$

▷ $C \rightarrow dC | da | dA$

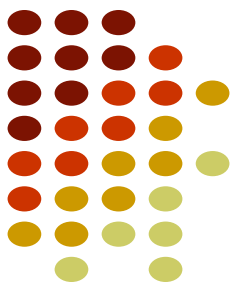
	Premier	Suivant
S	$\{a, b, c, d\}$	$\{\$ \}$
A	$\{a, \epsilon\}$	$\{b, c, d, \$ \}$
B	$\{b, c, \epsilon\}$	$\{d\}$
C	$\{d\}$	$\{\$ \}$

Structure

	Symboles terminaux				
Symboles Non terminaux	a	b	c	d	\$
S	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	
A	$A \rightarrow aA$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
B		$B \rightarrow bB$	$B \rightarrow cB$	$B \rightarrow \epsilon$	
C					

Approche descendante (Top-down)

Analyseur LL(1)



Soit la grammaire suivante :

▷ $S \rightarrow ABC$

▷ $A \rightarrow aA | \varepsilon$

▷ $B \rightarrow bB | cB | \varepsilon$

▷ $C \rightarrow dC | da | dA$

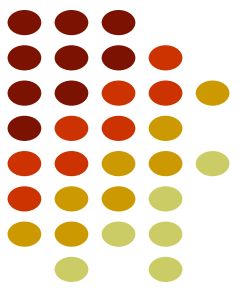
	Premier	Suivant
S	$\{a, b, c, d\}$	$\{\$ \}$
A	$\{a, \varepsilon\}$	$\{b, c, d, \$ \}$
B	$\{b, c, \varepsilon\}$	$\{d\}$
C	$\{d\}$	$\{\$ \}$

Structure

	Symboles terminaux				
Symboles Non terminaux	a	b	c	d	\$
S	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	
A	$A \rightarrow aA$	$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$
B		$B \rightarrow bB$	$B \rightarrow cB$	$B \rightarrow \varepsilon$	
C					

Approche descendante (Top-down)

Analyseur LL(1)



Soit la grammaire suivante :

- ▷ $S \rightarrow ABC$
- ▷ $A \rightarrow aA | \varepsilon$
- ▷ $B \rightarrow bB | cB | \varepsilon$
- ▷ $C \rightarrow dC | da | dA$

	Premier	Suivant
S	$\{a, b, c, d\}$	$\{\$ \}$
A	$\{a, \varepsilon\}$	$\{b, c, d, \$ \}$
B	$\{b, c, \varepsilon\}$	$\{d\}$
C	$\{d\}$	$\{\$ \}$

Structure

	Symboles terminaux				
Symboles Non terminaux	a	b	c	d	\$
S	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	$S \rightarrow ABC$	
A	$A \rightarrow aA$	$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$
B		$B \rightarrow bB$	$B \rightarrow cB$	$B \rightarrow \varepsilon$	
C				$C \rightarrow dC$ $C \rightarrow da$ $C \rightarrow dA$	

Grammaire non LL(1)

Approche descendante (Top-down)

Analyseur LL(1)

• Construction de la table d'analyse : Exemple

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid -TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid /FT' \mid \epsilon$
 $F \rightarrow (E) \mid nb$

	nb	+	-	*	/	(	)	\$
E								
E'								
T								
T'								
F								

$\text{PREMIER}(E) = \text{PREMIER}(T) = \{ (, nb \}$

$\text{PREMIER}(E') = \{ +, -, \epsilon \}$

$\text{PREMIER}(T) = \text{PREMIER}(F) = \{ (, nb \}$

$\text{PREMIER}(T') = \{ *, /, \epsilon \}$

$\text{PREMIER}(F) = \{ (, nb \}$

$\text{SUIVANT}(E) = \{ \$,) \}$

$\text{SUIVANT}(E') = \{ \$,) \}$

$\text{SUIVANT}(T) = \{ +, -,) \$ \}$

$\text{SUIVANT}(T') = \{ +, -,) \$ \}$

$\text{SUIVANT}(F) = \{ *, /,), +, -, \$ \}$

